

What is a Process?

A process is a program in execution. Think of it as a recipe (the program) being actively cooked (executed) by a chef (the CPU). It includes the program code, data, and the current state of execution (like where the chef is in the recipe). A process has its own memory space and resources, such as CPU time, files, and I/O devices.

Preemptive Systems

In preemptive systems, the operating system (OS) can interrupt a running process to give CPU time to another process. Imagine a chef cooking multiple dishes but switching between them to ensure none burn. This ensures fair CPU sharing, especially in multi-tasking environments. Example: Modern OS like Windows or Linux, where apps run simultaneously, and the OS decides when to switch.

Non-Preemptive Systems

In non-preemptive systems, a process keeps the CPU until it finishes or voluntarily gives it up (e.g., by waiting for I/O). Think of a chef finishing one dish completely before starting another. This can lead to delays if a process takes too long. Example: Older OS or simple embedded systems.

5

Process States

Process State Transition Diagram

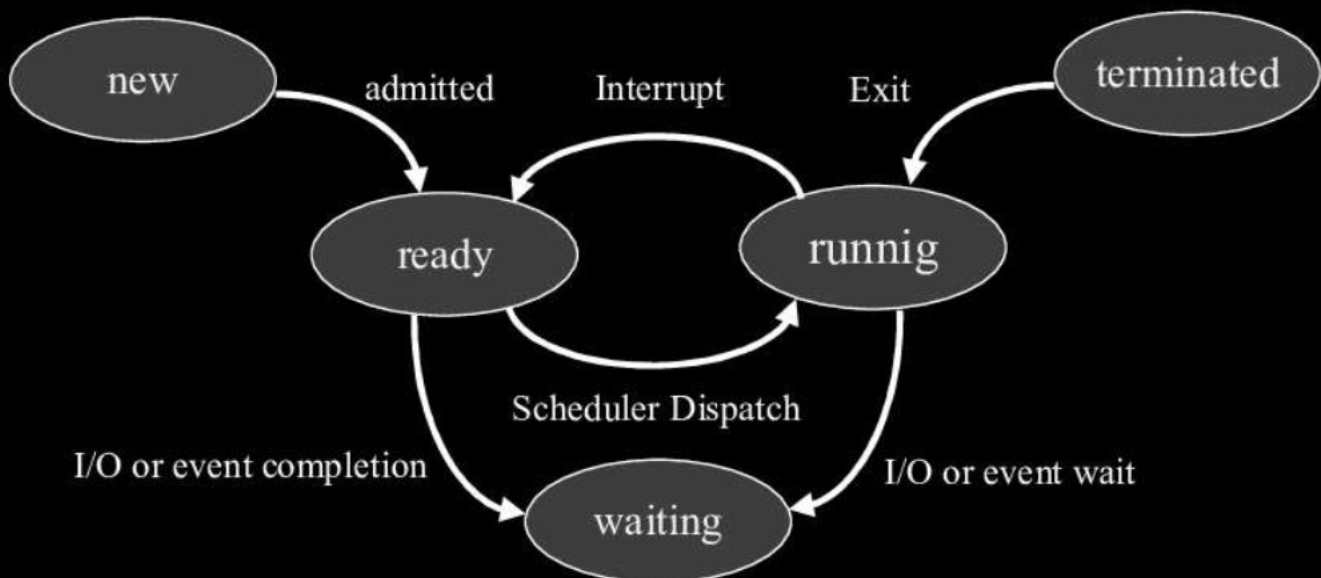
The process state transition diagram shows how a process moves between different states during its lifecycle. Think of it like a game character moving through levels (states) as it interacts with the game world (OS).

Example in Context of Preemption

Let's use a relatable example: a music streaming app (like Spotify) on your laptop, running in a preemptive OS. The app is a process that plays music, fetches playlists, and updates the UI.

- **New**: The app is launched but not yet ready to run (e.g., loading into memory).
- **Ready**: The app is loaded and waiting for CPU time to execute (e.g., ready to fetch a playlist).
- **Running**: The app is actively playing music or updating the screen.
- **Blocked**: The app pauses to wait for internet data (e.g., buffering a song).
- **Terminated**: The app closes completely.

In a preemptive system, the OS might interrupt the app (e.g., when you open a browser) to give CPU time to another process, moving the app from **Running** to **Ready**. This ensures smooth multitasking.



With Respect to Multi-Process OS

In a multi-process OS, multiple processes share the CPU. The OS uses a scheduler to decide which process gets the CPU, especially in preemptive systems. The diagram helps visualize how processes compete for CPU time and move between states based on OS decisions or process actions.

Process State Definitions

- **New State:** The process is being created and loaded into memory. Example: You click the Spotify icon, and the OS starts loading the app.
- **Ready State:** The process is ready to run but waiting for CPU allocation. Example: Spotify is loaded but waiting because the browser is using the CPU.
- **Running State:** The process is actively executing on the CPU. Example: Spotify is playing music or updating the playlist.
- **Waiting/Blocked State:** The process is paused, waiting for an event (e.g., I/O operation). Example: Spotify waits for a song to buffer from the internet.
- **Terminated State:** The process has finished execution or been killed. Example: You close Spotify, and it exits.

Process States Transition

These transitions describe how a process moves between states, either by its own actions or by the OS.

- **New to Ready:** The OS finishes loading the process into memory, making it ready to run. Example: Spotify is fully loaded and waits for CPU.
- **Ready to Running:** The OS assigns the CPU to the process. Example: Spotify gets CPU time to play a song.
- **Running to Terminated:** The process completes or is terminated by itself. Example: Spotify closes after you exit.
- **Running to Blocked:** The process needs to wait for an event (e.g., I/O). Example: Spotify pauses to buffer a song.
- **Running to Ready:** The OS interrupts the process (preemption) to give CPU to another process. Example: Spotify pauses briefly when you open a browser.
- **Blocked to Ready:** The event the process was waiting for completes, and it's ready for CPU again. Example: Spotify finishes buffering and waits for CPU.

Rest of All Transitions Are Done by OS Only: Transitions like **Ready to Running** or **Running to Ready** (in preemption) are controlled by the OS scheduler, not the process.

What Actions Can a Process Take by Itself from Running State?

From the **Running** state, a process can voluntarily transition to:

- **Terminated State:** The process finishes or exits. Example: You close Spotify.
- **Waiting/Blocked State:** The process waits for an event, like I/O. Example: Spotify waits for internet data.

These are **voluntary transitions** because the process initiates them (e.g., by requesting I/O or exiting).

Let's complete the section in your notes for the case where **N processes** and **M CPUs** are given, with $M > N$. I'll provide the minimum and maximum number of processes in each state (Running, Ready, Blocked) and ensure consistency with the format you provided for the case where $M < N$.

Question: N Processes, M CPUs ($M > N$), Min and Max Processes in Each State

If there are **N processes** and **M CPUs** (where $M > N$), the number of processes in each state depends on the system's workload and scheduling. Since there are more CPUs than processes, all processes could potentially run simultaneously, but the actual distribution depends on their state.

- **Running State:**
 - **Minimum:** 0 (if all processes are blocked or none are scheduled to run).
 - **Maximum:** N (since $M > N$, all N processes could run simultaneously if they are not blocked, as there are enough CPUs).
- **Ready State:**
 - **Minimum:** 0 (if all processes are running or blocked).
 - **Maximum:** N (if all processes are ready to run but none are scheduled yet, though this is rare since $M > N$ allows all processes to run).
- **Blocked State:**

- **Minimum:** 0 (if no processes are waiting for I/O or events).
- **Maximum:** N (if all processes are waiting for I/O or events and none are running or ready).

Example: If $N = 3$ processes and $M = 5$ CPUs:

- **Running:** Min = 0, Max = 3.
- **Ready:** Min = 0, Max = 3.
- **Blocked:** Min = 0, Max = 3.

Question: N Processes, M CPUs ($M > N$), Min and Max Processes in Each State

If there are N processes and M CPUs (where $M > N$), the number of processes in each state depends on the system's workload and scheduling. Since there are more CPUs than processes, all processes could potentially run simultaneously.

- **Running State:**
 - **Minimum:** 0 (if all processes are blocked or none are scheduled to run).
 - **Maximum:** N (since $M > N$, all N processes could run simultaneously if not blocked).
- **Ready State:**
 - **Minimum:** 0 (if all processes are running or blocked).
 - **Maximum:** N (if all processes are ready but none are scheduled yet, though this is rare since $M > N$).
- **Blocked State:**
 - **Minimum:** 0 (if no processes are waiting for I/O or events).
 - **Maximum:** N (if all processes are waiting for I/O or events).

Example: If $N = 3$ processes and $M = 5$ CPUs:

- **Running:** Min = 0, Max = 3.
- **Ready:** Min = 0, Max = 3.
- **Blocked:** Min = 0, Max = 3.

Question: N Processes, M CPUs ($M < N$), Min and Max Processes in Each State

If there are N processes and M CPUs (where $M < N$), the number of processes in each state depends on the system's workload and scheduling.

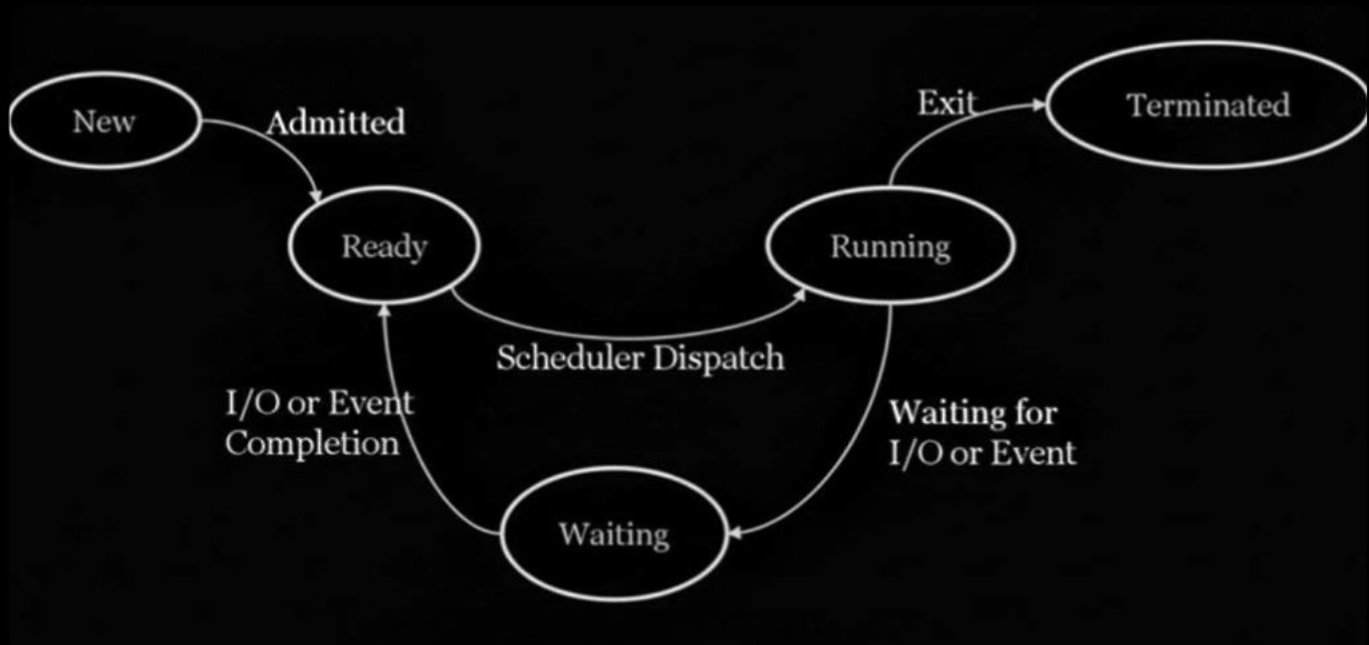
- **Running State:**
 - **Minimum:** 0 (if all processes are blocked or ready, and no CPU is allocated).
 - **Maximum:** M (each CPU can run one process at a time, so at most M processes are running).
- **Ready State:**
 - **Minimum:** 0 (if all processes are running or blocked).
 - **Maximum:** $N - M$ (if M processes are running, the remaining $N - M$ are ready or blocked).
- **Blocked State:**
 - **Minimum:** 0 (if no processes are waiting for I/O or events).
 - **Maximum:** $N - M$ (if M processes are running, all others could be blocked).

Example: If $N = 5$ processes and $M = 2$ CPUs:

- **Running:** Min = 0, Max = 2.
- **Ready:** Min = 0, Max = 3 ($5 - 2$).
- **Blocked:** Min = 0, Max = 3.



Process States: Non-Preemptive



In non-preemptive systems, a process in the **Running** state keeps the CPU until it:

- Completes (**Running to Terminated**).
- Requests I/O or an event (**Running to Blocked**).
- Voluntarily yields the CPU (if supported, **Running to Ready**).

Key differences from preemptive systems:

- The OS cannot interrupt a running process.
- Transitions like **Running to Ready** are rare unless the process explicitly yields.
- The process state diagram is similar, but transitions depend more on the process's actions than OS scheduling.

Example: In a non-preemptive system, if Spotify is playing a song (Running), it keeps the CPU until it finishes or waits for buffering (Blocked). Other apps (e.g., a browser) must wait.



CPU vs I/O Bound Process

Simple Definition for Both with Examples

- **CPU-Bound Process:**

- **Definition:** A process that primarily uses the CPU for computations and requires minimal I/O.
- **Example:** A video rendering program (e.g., Adobe Premiere) performing heavy calculations to encode a video. It keeps the CPU busy with math operations.

- **I/O-Bound Process:**

- **Definition:** A process that spends most of its time waiting for I/O operations (e.g., reading/writing to disk or network).
- **Example:** A file download manager waiting for data from the internet. It frequently goes to the **Blocked** state.

Key Difference: CPU-bound processes need more CPU time, while I/O-bound processes spend more time waiting for external events.